



Avance del Proyecto Final 3

Curso:

Base de Datos I

Implementación de una Base de Datos para la Clínica Ricardo Palma

Integrantes:

Bryan Alexander Luque Serna

Diego Alvaro Ayay Palomino

Misael Martin Alarcon Maza

Isabella Valentyna Segura Pizarro

Docente:

Danila Raquel Neira Sacaski

Sección:

31872

2026

Tabla de contenido

Implementación de una Base de Datos para la Clínica Ricardo Palma	4
1. Funciones Agregadas.....	4
1.1. Count.....	4
1.2. Sum.....	4
1.3. Avg.....	4
1.4. Max.....	4
1.5. Min.....	5
1.6. Group.....	5
1.7. Having.....	5
2. Subconsultas.....	6
2.1. In.....	6
2.2. Exists.....	6
3. Funciones De Fecha.....	7
3.1. Day.....	7
3.2. Month.....	7
3.3. Year.....	7
3.4. Datediff	8
3.5. Dateadd	8
4. Funciones De Cadena.....	10
4.1. Concat.....	10
4.2. Len.....	10
4.3. Substring.....	10
4.4. Lower.....	11
4.5. Upper.....	11
4.6. Ltrim.....	12
4.7. Rtrim.....	12
5. Funciones De Conversión.....	13
5.1. Cast.....	13
5.2. Convert.....	13
6. Transact Sql.....	14
6.1. Funciones.....	14
6.2. Procedimientos Almacenados.....	15
6.3. Condicional IF.....	15
6.4. Programacion De Cursores.....	16

Implementación de una Base de Datos para la Clínica Ricardo Palma

1. Funciones Agregadas

1.1. Count

-- COUNT(): cuenta el número de registros

```
SELECT COUNT(*) AS total_pacientes
```

```
FROM paciente;
```

-- Devuelve cuántos pacientes hay en la tabla

Resultado:

Result Grid			Filter Rows:		Export:		Wrap Cell Content:	
#	total_paciente							
1	20							

1.2. Sum

-- SUM(): Suma el total de las ventas netas registradas

```
SELECT SUM(precio_netto) AS total_ventas
```

```
FROM detalle_boleta;
```

-- Devuelve el monto total de todas las ventas netas

Resultado:

#	total_ventas							
1	2940.00							

1.3. Avg

-- AVG(): Calcula el precio promedio de los servicios

```
SELECT AVG(precio_unitario) AS precio_promedio
```

```
FROM detalle_boleta;
```

-- Devuelve el precio promedio de los servicios registrados.

Resultado:

#	precio_promedio							
1	147.000000							

1.4. Max

-- MAX(): Devuelve el valor máximo de una columna.

```
SELECT MAX(precio_netto) AS mayor_venta
```

```
FROM detalle_boleta;
```

-- Devuelve la venta con el precio neto más alto

Resultado:

#	mayor_venta
1	320.00

1.5. Min

-- MIN(): Devuelve el valor mínimo de una columna.

```
SELECT MIN(descuento) AS menor_descuento
FROM detalle_boleta;
```

-- Devuelve el descuento más bajo registrado, si no hay descuento es 0

Resultado:

#	menor_descuento
1	0.00

1.6. Group

-- AGRUPAR las citas por estado y contar cuántas existen en cada uno

```
SELECT estado_cita, COUNT(*) AS total_citas
FROM cita
GROUP BY estado_cita;
```

Resultado:

#	estado_cita	total_citas
1	Pendiente	16
2	Atendida	10
3	Cancelada	2

1.7. Having

-- MOSTRAR solo los estados que tengan más de 5 citas registradas

```
SELECT estado_cita, COUNT(*) AS total_citas
FROM cita
GROUP BY estado_cita
HAVING COUNT(*) > 5;
```

Resultado:

#	estado_cita	total_citas
1	Pendiente	16
2	Atendida	10

2. Subconsultas

2.1. In

-- MOSTRAR los consultorios en donde la especialidad este "Activo"

```
SELECT *
FROM consultorio
WHERE id_especialidad IN (SELECT id_especialidad FROM especialidad WHERE estado =
'Activo');
```

Resultado:

#	id_consultorio	id_especialidad	nombre_consultorio	numero_sala	piso	fecha_creacion	fecha_modificacion
1	1	1	Consultorio Medicina Interna A	101	1	2026-06-25 10:00:59	2026-06-25 10:00:59
2	2	1	Consultorio Medicina Interna B	102	1	2026-06-25 10:00:59	2026-06-25 10:00:59
3	3	2	Consultorio Pediatría A	103	1	2026-06-25 10:00:59	2026-06-25 10:00:59
4	4	2	Consultorio Pediatría B	201	2	2026-06-25 10:00:59	2026-06-25 10:00:59
5	5	3	Consultorio Ginecología	202	2	2026-06-25 10:00:59	2026-06-25 10:00:59
6	6	4	Consultorio Cardiología A	301	3	2026-06-25 10:00:59	2026-06-25 10:00:59
7	7	4	Consultorio Cardiología B	302	3	2026-06-25 10:00:59	2026-06-25 10:00:59
8	8	5	Consultorio Dermatología	401	4	2026-06-25 10:00:59	2026-06-25 10:00:59
9	9	6	Consultorio Oftalmología	402	4	2026-06-25 10:00:59	2026-06-25 10:00:59
10	10	1	Consultorio Medicina Interna C	501	5	2026-06-25 10:00:59	2026-06-25 10:00:59
11	11	2	Consultorio Pediatría C	502	5	2026-06-25 10:00:59	2026-06-25 10:00:59
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

2.2. Exists

-- MOSTRAR los pacientes que tengan al menos una cita registrada

```
SELECT *
FROM paciente p
WHERE EXISTS (
    SELECT 1
    FROM cita c
    WHERE c.id_paciente = p.id_paciente
);
```

Resultado:

#	id_pa	codigo_pac	dni	nombres	apellidos	tipo_sangre	fecha_nacimiento	genero_nacimiento	direccion_cliente	telefono
1	1	PAC001	48392017	Juan Alberto	Pérez Mendoza	A+	1990-05-10	M	Av. Siempre Viva 123	942 12
2	2	PAC002	71645829	Maria Fernanda	Lopez Ramirez	A+	2000-08-15	F	Av Lima 456	945 21
3	3	PAC003	30587146	Carlos Alberto	Rojas Vargas	B+	1995-03-20	M	Jr Arequipa 200	912 33
4	4	PAC004	92816473	Ana Lucia	Torres Mendoza	AB+	2002-01-11	F	Av Brasil 900	934 22
5	5	PAC005	56473821	Luis Enrique	Gomez Silva	O-	1997-07-19	M	Av Colonial 120	956 77
6	6	PAC006	14725836	Sofia Valentina	Diaz Herrera	A-	2004-09-01	F	Jr Tacna 880	923 44
7	7	PAC007	83920571	Pedro Antonio	Castillo Flores	B-	1993-04-14	M	Av Marina 500	965 21
8	8	PAC008	67294158	Camila Alejandra	Mendoza Ruiz	AB-	2001-12-22	F	Av Grau 101	941 28
9	9	PAC009	25168394	Miguel Angel	Salas Paredes	O+	1999-10-18	M	Av Mexico 450	978 45
10	10	PAC010	79431625	Valeria Nicole	Ruiz Torres	A+	2003-02-03	F	Jr Cusco 741	932 14

3. Funciones De Fecha

3.1. Day

/* Función DAY*/

-- MOSTRAR todas las citas para el DIA 22

SELECT id_cita,fecha,hora

FROM cita

WHERE DAY(fecha) = 22;

Resultado:

#	id_cita	fecha	hora
1	7	2026-07-22	08:30:00
2	8	2026-07-22	15:30:00
3	9	2026-07-22	10:00:00
4	10	2026-07-22	22:15:00
*	NULL	NULL	NULL

3.2. Month

/* Función MONTH*/

-- MOSTRAR todas las citas para el MES de JULIO

SELECT id_cita,fecha,hora

FROM cita

WHERE MONTH(fecha) = 7;

Resultado:

#	id_cita	fecha	hora
1	1	2026-07-20	08:00:00
2	2	2026-07-20	15:00:00
3	3	2026-07-20	22:30:00
4	4	2026-07-21	09:00:00
5	5	2026-07-21	16:00:00
6	6	2026-07-21	23:00:00
7	7	2026-07-22	08:30:00
8	8	2026-07-22	15:30:00
9	9	2026-07-22	10:00:00
10	10	2026-07-22	22:15:00

3.3. Year

/* Función YEAR */

-- Mostrar las citas para el año 2026

SELECT id_cita,fecha,hora

FROM cita

WHERE YEAR(fecha) = 2026;

Resultado:

#	id_cita	fecha	hora
1	1	2026-07-20	08:00:00
2	2	2026-07-20	15:00:00
3	3	2026-07-20	22:30:00
4	4	2026-07-21	09:00:00
5	5	2026-07-21	16:00:00
6	6	2026-07-21	23:00:00
7	7	2026-07-22	08:30:00
8	8	2026-07-22	15:30:00
9	9	2026-07-22	10:00:00
10	10	2026-07-22	22:15:00

3.4. Datediff

/* Función DATEDIFF */

-- MOSTRAR los días que faltan para que se lleva a cabo una cita

```
SELECT id_cita, fecha , DATEDIFF(fecha, NOW()) AS dias_faltantes
FROM cita;
```

Resultado:

#	id_cita	fecha	dias_faltantes
1	1	2026-07-20	22
2	2	2026-07-20	22
3	3	2026-07-20	22
4	4	2026-07-21	23
5	5	2026-07-21	23
6	6	2026-07-21	23
7	7	2026-07-22	24
8	8	2026-07-22	24
9	9	2026-07-22	24
10	10	2026-07-22	24

3.5. Dateadd

/* Función DATE_ADD */

-- PROGRAMAR una cita 5 días después desde que vamos a agendarla

```
SELECT id_cita, fecha, DATE_ADD(fecha, INTERVAL 5 DAY) AS Fecha_De_Cita
FROM cita;
```

Resultado:

#	id_cita	fecha ▼	Fecha_De_Cita
1	1	2026-07-20	2026-07-25
2	2	2026-07-20	2026-07-25
3	3	2026-07-20	2026-07-25
4	4	2026-07-21	2026-07-26
5	5	2026-07-21	2026-07-26
6	6	2026-07-21	2026-07-26
7	7	2026-07-22	2026-07-27
8	8	2026-07-22	2026-07-27
9	9	2026-07-22	2026-07-27
10	10	2026-07-22	2026-07-27

4. Funciones De Cadena

4.1. Concat

-- CONCAT(): Une nombres y apellidos para formar el nombre completo en una sola columna

```
SELECT CONCAT(nombres, ' ', apellidos) AS nombre_completo
```

```
FROM paciente;
```

Resultado:

#	nombre_completo
1	Ana Lucia Torres Mendoza
2	Camila Alejandra Mendoza Ruiz
3	Carlos Alberto Rojas Vargas
4	Daniela Cristina Paredes Castro
5	Diego Sebastian Herrera Cruz
6	Elena Maribel Ramos Delgado
7	Fernanda Isabel Cruz Medina
8	Gabriela Sofia Ortega Leon
9	Jorge Luis Reyes Campos
10	Jose Manuel Vargas Romero

4.2. Len

-- LENGTH(): Muestra la cantidad de caracteres del nombre del médico

```
SELECT nombres, LENGTH(nombres) AS longitud_nombre
```

```
FROM medico;
```

Resultado:

#	nombres	longitud_nombre
1	María	6
2	Luis Fernando	13
3	Miguel Angel	12
4	Javier Enrique	14
5	Fernando Jose	13
6	Ricardo Manuel	14
7	Patricia Elena	14
8	Andrea Lucia	12
9	Lucia Fernanda	14
10	Sandra Milagros	15

4.3. Substring

-- SUBSTRING(): Muestra los primeros 4 dígitos del DNI del paciente

```
SELECT dni, SUBSTRING(dni, 1, 4) AS primeros_digitos
```

```
FROM paciente;
```

Resultado:

#	dni	primeros_digitos
1	14725836	1472
2	17359486	1735
3	25168394	2516
4	29475813	2947
5	30587146	3058
6	36192584	3619
7	42068375	4206
8	48392017	4839
9	50827419	5082
10	51937628	5193

4.4. Lower

-- LOWER(): Convierte el correo a minúsculas

```
SELECT LOWER(correo) AS correo_minusculas
FROM medico;
```

Resultado:

#	correo_minusculas
1	andreacastillovega@ricardopalma.com
2	claudianavarroleon@ricardopalma.com
3	danielaparedessoto@ricardopalma.com
4	eduardoreyesgomez@ricardopalma.com
5	felipesanchez@ricardopalma.com
6	fernandoruizherrera@ricardopalma.com
7	javiersalasromero@ricardopalma.com
8	joseramossilva@ricardopalma.com
9	lauraespinoza@ricardopalma.com
10	luciamendozaruiz@ricardopalma.com

4.5. Upper

-- UPPER(): Convierte el nombre de la especialidad a mayúsculas

```
SELECT UPPER(nombre_especialidad) AS especialidad_mayusculas
FROM especialidad;
```

Resultado:

#	especialidad_mayusculas
1	CARDIOLOGIA
2	DERMATOLOGIA
3	GINECOLOGIA
4	MEDICINA INTERNA
5	NEUROLOGIA
6	OFTALMOLOGIA
7	PEDIATRIA
8	PSIQUIATRIA
9	TRAUMATOLOGIA

4.6. Ltrim

-- LTRIM(): Elimina los espacios en blanco al inicio de la dirección
 SELECT LTRIM(direccion_cliente) AS direccion_sin_espacios_izq
 FROM paciente;

Resultado:

#	direccion_sin_espacios_izq
1	Av. Siempre Viva 123
2	Av Lima 456
3	Jr Arequipa 200
4	Av Brasil 900
5	Av Colonial 120
6	Jr Tacna 880
7	Av Marina 500
8	Av Grau 101
9	Av Mexico 450
10	Jr Cusco 741

4.7. Rtrim

-- RTRIM(): Elimina los espacios en blanco al final del teléfono
 SELECT RTRIM(telefono) AS telefono_sin_espacios_der
 FROM recepcionista;

Resultado:

#	telefono_sin_espacios_der
1	987 452 163
2	954 218 736
3	978 364 125
4	943 625 781

5. Funciones De Conversión

5.1. Cast

-- CAST(): Convierte la fecha a tipo CHAR

```
SELECT id_cita, CAST(fecha AS CHAR) AS fecha_texto
FROM cita;
```

Resultado:

#	id_cita	fecha_texto
1	1	2026-07-20
2	2	2026-07-20
3	3	2026-07-20
4	4	2026-07-21
5	5	2026-07-21
6	6	2026-07-21
7	7	2026-07-22
8	8	2026-07-22
9	9	2026-07-22
10	10	2026-07-22

5.2. Convert

-- CONVERT(): Convierte la hora a tipo CHAR

```
SELECT id_cita, CONVERT(hora, CHAR) AS hora_texto
FROM cita;
```

Resultado:

#	id_cita	hora_texto
1	1	08:00:00
2	2	15:00:00
3	3	22:30:00
4	4	09:00:00
5	5	16:00:00
6	6	23:00:00
7	7	08:30:00
8	8	15:30:00
9	9	10:00:00
10	10	22:15:00

6. Transact Sql

6.1. Funciones

-- Función para contar citas por médico

```
DROP FUNCTION IF EXISTS citasPorMedico;
```

```
DELIMITER //
```

```
CREATE FUNCTION citasPorMedico(idMedico INT)
```

```
RETURNS INT
```

```
BEGIN
```

```
    DECLARE total INT;
```

```
    SELECT COUNT(*) INTO total
```

```
    FROM cita
```

```
    WHERE id_medico = idMedico;
```

```
    RETURN total;
```

```
END//
```

```
DELIMITER ;
```

```
SELECT nombres, apellidos, citasPorMedico(id_medico) AS total_citas
```

```
FROM medico;
```

Resultado:

#	nombres	apellidos	total_citas
1	María	Gómez Prado	2
2	Luis Fernando	Garcia Ramos	2
3	Miguel Angel	Torres Flores	2
4	Javier Enrique	Salas Romero	2
5	Fernando Jose	Ruiz Herrera	2
6	Ricardo Manuel	Flores Diaz	2
7	Patricia Elena	Diaz Castro	2
8	Andrea Lucia	Castillo Vega	1
9	Lucia Fernanda	Mendoza Ruiz	1
10	Sandra Milagros	Vega Paredes	1
11	Daniela Cristina	Paredes Soto	1
12	Rosa Maria	Herrera Ca...	1
13	Jose Antonio	Ramos Silva	1
14	Pedro Luis	Lopez Medina	1
15	Marco Aurelio	Vargas Torres	1
16	Victor Manuel	Salazar Rojas	1
17	Claudia Patricia	Navarro Leon	1
18	Monica Isabel	Campos Diaz	1
19	Eduardo Javier	Reyes Gomez	1
20	Victor Hugo	Castro Silva	1
21	Raul Ernesto	Ortega Ruiz	1
22	Felipe Juan	Sanchez M...	0

6.2. Procedimientos Almacenados

-- Mostrar todos los médicos de un turno

```
DROP PROCEDURE IF EXISTS mostrar_medicos_turno;
```

```
DELIMITER //
```

```
CREATE PROCEDURE mostrar_medicos_turno(IN p_turno VARCHAR(10))
BEGIN
```

```
    SELECT nombres, apellidos, turno
    FROM medico
    WHERE turno = p_turno;
```

```
END //
```

```
DELIMITER ;
```

```
CALL mostrar_medicos_turno("Tarde");
```

Resultado:

#	nombres	apellidos	turno
1	Fernando Jose	Ruiz Herrera	Tarde
2	Andrea Lucia	Castillo Vega	Tarde
3	Rosa Maria	Herrera Campos	Tarde
4	Pedro Luis	Lopez Medina	Tarde
5	Claudia Patricia	Navarro Leon	Tarde
6	Victor Hugo	Castro Silva	Tarde
7	Felipe Juan	Sanchez Mendoza	Tarde
8	Laura Luna	Espinoza Ramos	Tarde

6.3. Condicional IF

/--- IF ELSE ---/

```
DROP PROCEDURE IF EXISTS verificar_paciente_registrado;
```

-- Si queremos saber si un paciente ya está registrado en la base de datos

```
DELIMITER //
```

```
CREATE PROCEDURE verificar_paciente_registrado(IN p_dni INT)
```

```
BEGIN
```

```
    IF (EXISTS (SELECT 1
                FROM paciente
```

```
                WHERE dni = p_dni)) THEN
```

```
        SELECT codigo_paciente, CONCAT(nombres, ' ', apellidos) AS
```

```
Nombre_completo, fecha_nacimiento, 'Paciente Registrado!' AS ESTADO
```

```
        FROM paciente
```



```

WHERE p_dni = dni;
ELSE
    SELECT 'El paciente NO está registrado.' AS Estado;
END IF;
END//

```

```
DELIMITER ;
```

```
CALL verificar_paciente_registrado(70491376);
```

Resultado:

#	Estado
1	El paciente NO está registrado.

6.4. Programacion De Cursores

```
USE clinica_db;
```

```
DROP PROCEDURE IF EXISTS sp_listar_citas_pendientes;
```

```
DELIMITER //
```

```
CREATE PROCEDURE sp_listar_citas_pendientes()
BEGIN
```

```

    DECLARE v_paciente VARCHAR(200);
    DECLARE v_fecha DATE;
    DECLARE v_hora TIME;
    DECLARE fin INT DEFAULT FALSE;

```

```

    DECLARE cur CURSOR FOR
        SELECT CONCAT(p.nombres, ' ', p.apellidos),
               c.fecha,
               c.hora
        FROM cita c
        JOIN paciente p ON c.id_paciente = p.id_paciente
        WHERE c.estado_cita = 'Pendiente';

```

```

    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET fin = TRUE;

```

```
OPEN cur;
```

```

bucle: LOOP
    FETCH cur INTO v_paciente, v_fecha, v_hora;

```

```

    IF fin THEN
        LEAVE bucle;
    END IF;

```

```
SELECT CONCAT(
```



```
'Paciente: ', v_paciente,  
' Fecha: ', v_fecha,  
' Hora: ', v_hora  
) AS informacion;  
END LOOP;  
  
CLOSE cur;  
END //  
DELIMITER ;  
  
CALL sp_listar_citas_pendientes();
```

Resultado:

#	informacion
1	Paciente: Jose Manuel Vargas Romero Fecha: 2026-07-23 Hora: 08:00:00

◀ Result 1 ×

Result 2 ×

Result 3 ×

Result 4 ×

Result 5 ×